

Workshop Cheatsheet

Companion to the PyCon Italy 2026 workshop “Write Your First High-Performance GPU Kernel in Python!” — Triton primitives, GPU vocabulary, and the gotchas, roughly in the order you hit them: §0 orientation → §1–§9 the kernel → §10 go deeper → §11 every term we use. Full Triton API at triton-lang.org · `import triton; import triton.language as tl`.

§0 · 5-minute orientation

If you're new to GPU programming, read this first. Otherwise skip ahead.

SPMD: same program, many data chunks. A Triton kernel is a single piece of Python code that runs many times in parallel — each instance is called a *program*. You write the code for one program; the runtime launches a *grid* of them, each operating on a different slice of the input. The grid is one-, two-, or three-dimensional, and you choose its size at launch. (In [Flynn's taxonomy](#) terms: you write SPMD, the hardware runs SIMT — §10.)

Block: how much work one program owns. `BLOCK` is the number of elements one program is responsible for — often a power of two like 1024. You set it at launch (`kernel[grid](..., BLOCK=1024)`); the grid size is then `ceil(n / BLOCK)`. Inside one program, an operation on a BLOCK-sized vector happens across all BLOCK positions at once.

Lane: a parallel slot inside a program. Within one program, `a + b` on a BLOCK-sized vector executes across BLOCK *lanes* simultaneously — think NumPy vectorization, but in hardware. Most of the time you don't think about lanes; they only surface when some positions are out-of-bounds (§4 mask) or you make per-element choices (§7 `tl.where`).

Broadcasting: NumPy-style. Scalars stretch to match vector shapes: `pid * BLOCK + tl.arange(0, BLOCK)` produces a BLOCK-sized vector of offsets, because `pid * BLOCK` is a scalar and `tl.arange` returns a vector. For 2D tiles (§3), `[:, None]` turns a vector into a column and `[None, :]` turns it into a row; the two broadcast together into a 2D shape. Same rules as NumPy.

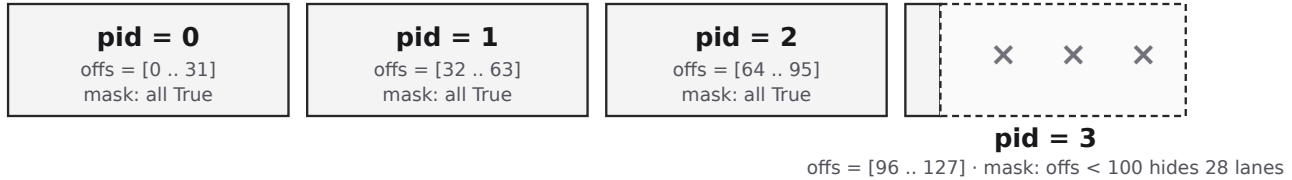
Why power-of-two `BLOCK`? Triton vectorizes operations down to *warps* — groups of 32 lanes that execute in lockstep on the hardware. A power-of-two `BLOCK` guarantees clean splits into warps. Non-powers-of-two work for some elementwise ops but break `tl.arange`, reductions, and `tl.dot`; round up with `triton.next_power_of_2` if your natural size isn't one.

If any primitive in §1–§9 still feels like a black box, that's §10's job — start with the relevant explainer before re-reading.

§1 · The mental model

You launch a [grid of programs](#); each owns one `BLOCK`-sized chunk; the last one masks its tail.

$$n = 100 \cdot \text{BLOCK} = 32 \cdot \text{grid} = (\text{cdiv}(100, 32),) = (4,)$$



```
grid = (triton.cdiv(n, BLOCK),)      # one program per chunk
kernel[grid](a, b, out, n, BLOCK=1024)
```

If `n` isn't a multiple of `BLOCK`, you have a tail — mask every load and store.

§2 · Kernel anatomy and launch

`@triton.jit` captures source; the first launch compiles; the second is a dict lookup.

```
@triton.jit
def kernel(a_ptr, b_ptr, out_ptr, n, BLOCK: tl.constexpr):
    pid = tl.program_id(axis=0)
    offs = pid * BLOCK + tl.arange(0, BLOCK)      # BLOCK must be a power of two
    mask = offs < n                               # tail-safe
    a = tl.load(a_ptr + offs, mask=mask)
    b = tl.load(b_ptr + offs, mask=mask)
    tl.store(out_ptr + offs, a + b, mask=mask)

compiled = kernel[grid](a, b, out, n, BLOCK=1024)  # returns a CompiledKernel
```

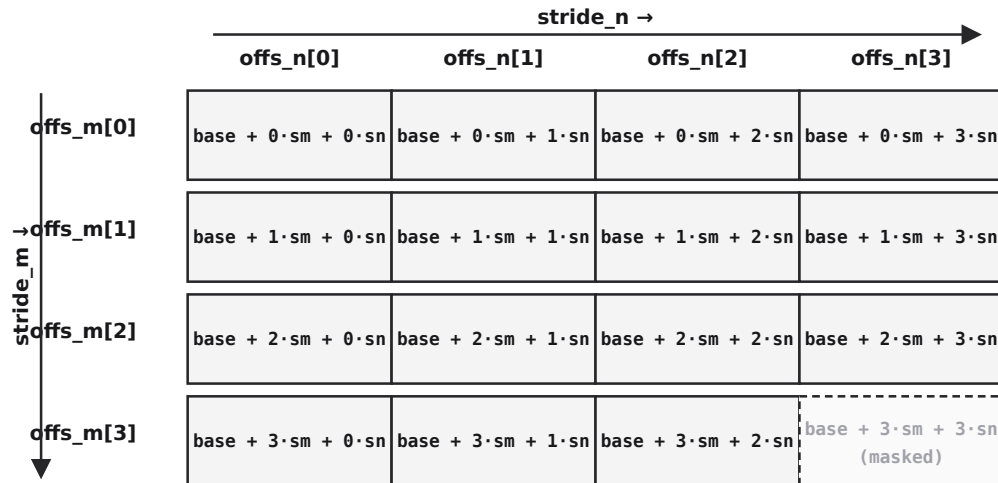
`grid` may also be `lambda meta: (triton.cdiv(n, meta['BLOCK']),)` — useful with autotune. Inspect IR via `compiled.asm['ttir' | 'ttgir' | 'llir' | 'ptx']`. If the natural size isn't a power of two, round up with `triton.next_power_of_2`.

Change a `tl.constexpr` value (e.g. `BLOCK`) and you trigger a fresh compile.

§3 · 2D tiles: broadcasting pointers

A 2D tile is a column vector times a row vector — pointers, not data.

2D tile of pointers · BLOCK_M = BLOCK_N = 4 · shape (M=4, N=4)



$\text{sm} = \text{stride_m}$ · $\text{sn} = \text{stride_n}$ · ptr math is typed: + offs advances offs · dtype.itemsize bytes

```
offs_m = pid_m * BLOCK_M + tl.arange(0, BLOCK_M)      # (M,)
offs_n = pid_n * BLOCK_N + tl.arange(0, BLOCK_N)      # (N,)
mask    = (offs_m[:, None] < M) & (offs_n[None, :] < N)
ptrs    = base + offs_m[:, None] * stride_m + offs_n[None, :] * stride_n
```

`[:, None]` / `[None, :]` broadcast like NumPy.

`ptr + offs` advances `offs · dtype.itemsize` bytes, not `offs` bytes.

§4 · Load, store, compute

Loads read masked positions and fill the misses with `other`; stores write masked positions and skip the misses.

`tl.load(a_ptr + offs, mask=mask, other=0.0) · n = 6, BLOCK = 8`

ptr = a_ptr+0 a_ptr+1 a_ptr+2 a_ptr+3 a_ptr+4 a_ptr+5 a_ptr+6 a_ptr+7

0.42	1.07	3.14	2.71	0.05	9.99	past n	past n
------	------	------	------	------	------	--------	--------

mask = offs < n ✓ ✓ ✓ ✓ ✓ ✗ ✗

`[0.42, 1.07, 3.14, 2.71, 0.05, 9.99, 0.0, 0.0]`

loaded vector · masked lanes filled with other=0.0

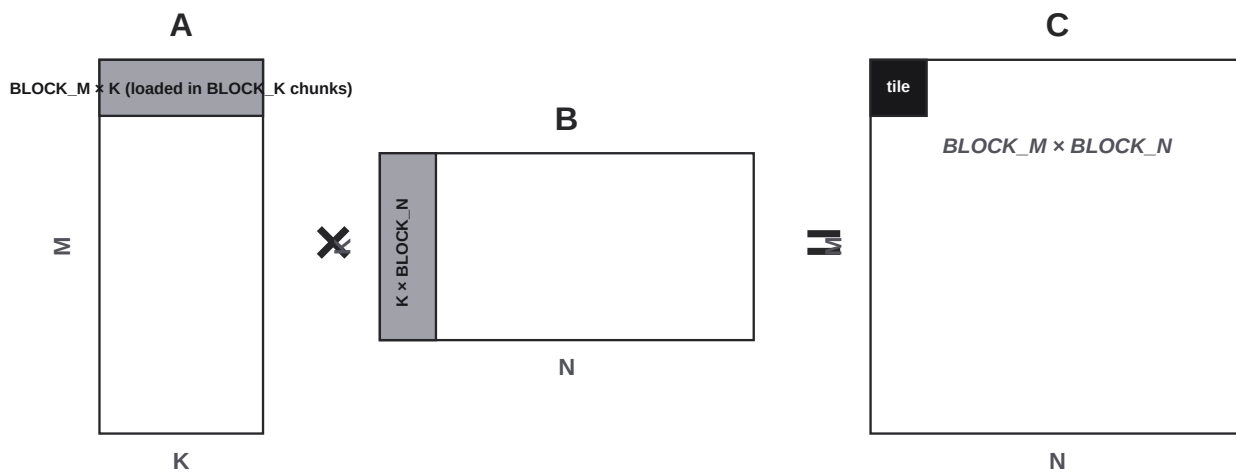
```
x = tl.load(ptr + offs, mask=mask, other=0.0)      # masked lanes return `other`
tl.store(ptr + offs, value, mask=mask)
```

Elementwise (broadcast like NumPy): `+`, `-`, `*`, `/`, `**`, `tl.exp`, `tl.log`, `tl.sqrt`, `tl.rsqrt`, `tl.sigmoid`, `tl.abs`, `tl.maximum`, `tl.minimum`, `tl.where(cond, a, b)`. Reductions (collapse one axis): `tl.sum`, `tl.max`, `tl.min`, `tl.argmax`, `tl.cumsum`. For [strided](#)/repeated 2D access: `tl.make_block_ptr` + `tl.advance`.

Forgetting `mask=` past the tail is the #1 way to corrupt memory.

§5 • `tl.dot` — tensor cores in a tile

One program computes one `BLOCK_M × BLOCK_N` tile of C by sweeping K with the accumulator in registers.



One program computes one tile of C · K-loop iterates in BLOCK_K chunks · accumulator (acc) stays in registers

```
acc = tl.zeros((BLOCK_M, BLOCK_N), dtype=tl.float32)
for k in range(0, K, BLOCK_K):
    a_tile = tl.load(a_ptrs)           # (BLOCK_M, BLOCK_K)
    b_tile = tl.load(b_ptrs)           # (BLOCK_K, BLOCK_N)
    acc = tl.dot(a_tile, b_tile, acc=acc) # FP32 accumulator
    a_ptrs += BLOCK_K * stride_ak
    b_ptrs += BLOCK_K * stride_bk
```

[Tensor cores](#) need FP16/BF16 inputs and `M, N, K ≥ 16` — otherwise you fall back to FP32 FMA.

§6 • What `@triton.jit` actually compiles

Your [Python](#) becomes Triton-IR → TritonGPU-IR → LLVM IR → PTX → SASS — cached by specialization key.



Cached in `~/.triton/cache/` (override with `TRITON_CACHE_DIR`). Cache key includes dtypes + every `tl.constexpr` value + pointer alignment hints — change any one, fresh compile.

Source edit not taking effect? `rm -rf ~/.triton/cache/`.

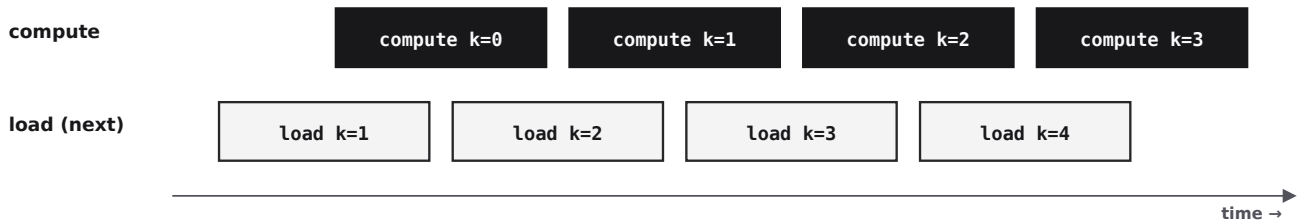
§7 · Control flow, types, atomics

Python `if` is for compile-time scalars; per-lane decisions use `tl.where`.

```
y = tl.where(x > 0, x, 0.0)          # per-lane choice
for k in tl.static_range(0, NUM_TILES): # unrolled at compile time
    ...
```

Dtypes: `tl.float32` / `float16` / `bfloat16` / `int32` / `int64` / `float8_e4m3` / `float8_e5m2` (FP8 needs sm_90+). Cast explicitly: `x.to(tl.float32)`. Atomics for cross-program writes (histograms, scatter): `tl.atomic_add(ptr + offs, value, mask=mask)`, also `tl.atomic_max`, `tl.atomic_min`, `tl.atomic_xchg`, `tl.atomic_cas`. Slow; prefer per-block reduction + one writeback.

`num_stages = 2` · one tile loads while the previous tile computes



`num_stages = 2` overlaps the next tile's load with the current tile's compute (see §8 to tune it).

`tl.static_range` unrolls and bloats the kernel — keep it under ~16 iterations.

§8 · Autotune

Compile every config, benchmark on the first call per `key=`, cache the winner.

First call with new `key=(M, N, K)` → compile each config (parallel) → benchmark → cache the winner.

config	compile	benchmark	measured
BLOCK = (64, 64, 32) num_warps = 4	<button>compile</button>	<button>benchmark</button>	2.10 μ s
BLOCK = (128, 128, 32) num_warps = 8	<button>compile</button>	<button>benchmark</button>	1.42 μ s ← winner
BLOCK = (128, 64, 64) num_warps = 4	<button>compile</button>	<button>benchmark</button>	1.78 μ s

→ **cache[key]**

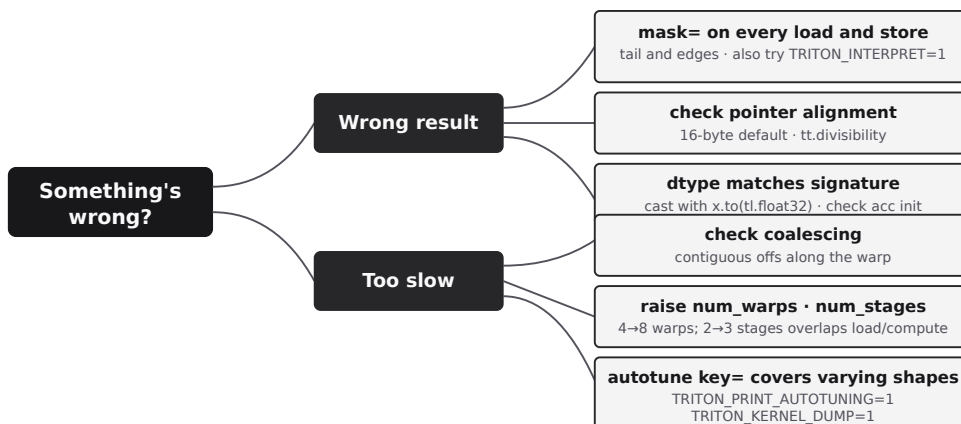
```
@triton.autotune(
    configs=[
        triton.Config({'BLOCK_M': 64, 'BLOCK_N': 64, 'BLOCK_K': 32}, num_warps=4),
        triton.Config({'BLOCK_M': 128, 'BLOCK_N': 128, 'BLOCK_K': 32}, num_warps=8),
        triton.Config({'BLOCK_M': 128, 'BLOCK_N': 64, 'BLOCK_K': 64}, num_warps=4),
    ],
    key=['M', 'N', 'K'],          # re-tune when these change
)
@triton.jit
def matmul_kernel(...): ...
```

`num_warps` (1–16): warps per program — more = more parallelism, more register pressure. `num_stages` (2–4): software-pipeline depth (see §7). For configs computed from runtime args (instead of benchmarked), use `triton.heuristics({'BLOCK': lambda meta: triton.next_power_of_2(meta['N'])})`.

Without `key=`, the first winner sticks forever — even when `M`, `N`, `K` change.

§9 · Debugging and gotchas

Symptom decides the knob. Helpers worth knowing: `tl.static_assert(BLOCK >= 16)` for compile-time checks; `tl.device_print("pid", pid, "x", x)` to log from the kernel (slow, strip before shipping).



Always true: power-of-two tiles · always mask the tail · no `print`, no allocations, no recursion inside a kernel · pointers default to 16-byte alignment.

§10 · Go deeper

The cheatsheet shows you *what* to write; these explainers on abhik.ai/concepts cover *why* the hardware and compiler behave this way. Read one when a primitive above still feels like a black box.

- [Flynn's Classification \(SISD / SIMD / MISD / MIMD → SIMT / SPMD\)](#) — §0. Why we describe Triton as SPMD, not SIMD or SIMT. SPMD is the level *you* write at; SIMT is what the warp scheduler actually does; SIMD is the CPU-vector cousin. Knowing which level a sentence lives at is how you read GPU papers without getting lost.
- [GPU Streaming Multiprocessor \(SM\)](#) — §1–2. “A grid of programs” is the abstraction; the SM is the physical core that runs your blocks, holds their registers, and caps occupancy. The reason `num_warps` and tile size trade off against each other.
- [GPU Memory Hierarchy](#) — §1, §5. Registers → shared memory → L2 → HBM, and the latency cliff between each tier. Tiling exists to climb data up this hierarchy once and reuse it.
- [High Bandwidth Memory \(HBM\)](#) — §4. Every uncached `tl.load` / `tl.store` is an HBM transaction. This is the bandwidth ceiling your kernel is measured against — and why contiguous, coalesced access wins.
- [Memory Access Patterns: Sequential vs Strided](#) — §3–4. Why the stride in your pointer arithmetic decides whether a load is one coalesced transaction or many — a 10×-plus swing in effective bandwidth.
- [Tensor Cores](#) — §5. The matrix-multiply hardware `tl.dot` targets: what FP16/BF16 inputs buy you, why `M, N, K ≥ 16`, and the FP32-FMA fallback when you miss those rules.
- [Python Bytecode Compilation](#) — §6. How Python source becomes something executable at all — the mental model that makes Triton's Python → IR → PTX → SASS pipeline click.

§11 · Glossary

Every technical term the slides, cheatsheet body, or notebooks use — defined for a reader with Python/NumPy and zero prior CUDA. Sections track the workshop arc; cross-references like §5 point at the relevant cheatsheet section, [→ deep dive] points at abhik.ai.

EXECUTION MODEL

- **kernel** — a single function that runs in parallel across thousands of GPU lanes. You write one body; the runtime instantiates a *grid* of them. In Triton it's any function tagged `@triton.jit`. (Naming collision: ML papers also call a 3×3 convolution filter a “kernel” — unrelated.)
- **thread** — the hardware-level name for a “lane”: the smallest independent unit of GPU execution. Triton hides threads behind BLOCK-sized vectors; they resurface when you read [PTX/SASS](#) or tune `num_warps`.
- **warp (Part 1)** — 32 [threads](#) on the same [SM](#) that execute the same instruction every cycle in [lockstep](#). Triton's “lane” within a program maps onto a warp thread once the [kernel](#) is compiled. The reason `BLOCK` must be a power of two.
- **SPMD (§0)** — *Single Program, Multiple Data*. You write the body of one program; the runtime launches a grid of them, each operating on a different data slice. The level Triton exposes to you. → [Flynn's](#)
- **SIMT (Part 1)** — *Single Instruction, Multiple Threads*. How the hardware actually runs a [warp](#). Lanes that branch differently take turns serially, not in parallel. The level *below* [SPMD](#).

- **lockstep (Part 1)** — all 32 lanes of a [warp](#) execute the same instruction in the same cycle. The cost of this design is [divergence](#).
- **divergence (Part 1)** — when [threads](#) in a [warp](#) follow different code paths; the warp must run each path while the other lanes sit idle. The reason `tl.where` (per-lane select) is faster than a Python `if` over per-lane data.
- **occupancy (Part 1)** — how many [warps](#) live on an [SM](#) at once. Higher occupancy hides memory latency better, but is capped by [registers](#) + [shared memory](#) per program — which is why a bigger tile isn't automatically faster.
- **latency hiding (Part 1)** — having enough [warps](#) resident that, while one warp waits ~400 cycles for [HBM](#), another can run. The reason GPUs need high [occupancy](#) at all.

HOST VS DEVICE (CUDA BASICS)

- **host** — the CPU process that drives the GPU: allocates buffers, copies data in/out, schedules [kernels](#).
- **device** — the GPU itself, with its own memory ([HBM](#)). Cannot read CPU memory directly; data crosses the PCIe bus.
- **kernel launch** — the [host](#)-side call (`kernel[grid](args)` in Triton, `kernel<<<...>>>(args)` in CUDA C++) that runs a [kernel](#) on the [device](#). First launch compiles (hundreds of ms); subsequent launches with the same [specialization](#) are a dict lookup (μ s).
- **CUDA (Parts 1, 4)** — NVIDIA's GPU compute stack: the C++ API, the [driver](#), the runtime, and the [PTX](#) instruction set all live under this umbrella. Triton compiles down through CUDA without you ever writing CUDA C++.
- **CuPy / `RawKernel` (Part 1)** — Python $\leftrightarrow$ CUDA bridge used in notebook 01. You write the [kernel](#) as a CUDA C++ string; CuPy compiles it via `nvcc` and lets you launch it from Python. Demonstrates “kernel = string” before Triton hides the string.
- **CUDA stream (out of scope; mentioned)** — an ordered queue of GPU operations. Different streams can overlap (compute on one while another copies). Not used in the workshop.

MEMORY HIERARCHY

- **HBM / global memory / device memory** — the GPU's off-chip DRAM (320 GB/s on [T4](#)). Visible to every program; high latency (~400 cycles); the only place big tensors fit. Every uncached `tl.load` / `tl.store` hits HBM. [→ deep dive](#)
- **L2 cache** — on-die cache shared by all [SMs](#) (~6 MB on [T4](#)). Sits between [HBM](#) and [L1](#); catches repeated reads from neighboring programs. `GROUP_SIZE_M` exists to exploit it.
- **L1 cache** — per-[SM](#) cache (~64 KB on [T4](#), physically shared with [shared memory](#)). First hop after a [register](#) miss.
- **shared memory** — on-chip scratchpad addressable by every [thread](#) in a program (~48 KB per [SM](#) on [T4](#)). [Register](#)-tier latency. Triton manages it implicitly when you do `tl.dot` or load a tile.
- **register** — per-thread fast storage; ~64 K 32-bit registers per [SM](#), split among resident [warps](#). The fastest tier. Where the matmul [accumulator](#) lives.

- **register pressure** (*Part 4*) — [registers](#) used per program. Cap ~255 per [thread](#); the [SM](#)-wide pool is fixed. High pressure means fewer programs fit on an SM → lower [occupancy](#) → less [latency hiding](#).
- **streaming multiprocessor (SM)** — a physical GPU core. [T4](#) has 40 SMs; each holds its resident [warps](#), their [registers](#), and its [shared memory](#). The unit you actually launch *onto*. → [deep dive](#)
- **contiguous** (*Part 2*) — a tensor whose elements lie in memory order, no skipped bytes. Row-major contiguous means neighboring lanes load neighboring bytes — the precondition for [coalescing](#).
- **stride** (*Parts 2, 4*) — byte offset between consecutive elements along a dimension. Row-major `(M, N): stride_m = N * itemsize, stride_n = itemsize`. Strided access (e.g., reading a column from a row-major matrix) is the #1 way to lose ~10× bandwidth.
- **coalesced** (*Parts 2, 4*) — 32 lanes of a [warp](#) load 32 contiguous 4-byte values in *one* [HBM](#) transaction instead of 32. Worth ~10–30× over non-coalesced. `tl.arange(0, BLOCK)` over a [contiguous](#) tensor coalesces for free. → [deep dive](#)

CUDA CONTRAST (PART 1 CUPY SLIDE ONLY)

- `tid / blockIdx / blockDim / threadIdx` (*Part 1*) — CUDA built-ins shown in the [RawKernel](#) slide. Triton's `pid ≈ blockIdx`, `BLOCK ≈ blockDim`, and `tl.arange(0, BLOCK)` does the `threadIdx` - per-program math for you — that's the abstraction Triton buys.

COMPILATION PIPELINE

- **JIT (just-in-time)** (*Part 2*) — compilation that happens on first call, once dtypes / shapes / `tl.constexpr` values are known. Triton (and PyTorch's `torch.compile`) work this way. Cost: first call is slow. Win: the [kernel](#) is specialized to your exact inputs.
- **AOT (ahead-of-time)** (*Part 2*) — compilation during build, before any input is known. C, Rust, CUDA C++ via `nvcc`. Win: no first-call latency. Cost: the [kernel](#) must handle every input shape generically.
- **driver** (*Part 2*) — the NVIDIA kernel-mode component that loads [PTX](#), [JITs](#) it to [SASS](#) for your specific GPU, manages memory, and runs kernels. The lower half of the JIT chain (Triton → PTX, driver → SASS).
- `@triton.jit` (§2, §6) — the decorator. Captures the function's Python source, compiles on first [launch](#), caches by [specialization key](#).
- `tl.constexpr` (§2) — type annotation marking a parameter as a *compile-time constant*. The compiler bakes the value into the IR (unrolling loops, sizing tiles). Changing it forces a fresh compile.
- **predicate / predication** (*Part 2*) — a 1-bit per-lane flag (`@%p1` in [PTX](#)). A predicated instruction executes only for lanes whose flag is true. What `mask=` actually lowers to — *not* a runtime `if` branch.
- **scalarize** (*Part 2*) — the compiler turning a tile-level op into per-lane instructions. `tl.load(ptr, mask=...)` becomes 32 individual loads gated by 32 [predicates](#).

- **SSA (Static Single Assignment) (Part 2)** — IR form where every variable is assigned *exactly once*. A source-level reassignment like `x = x + 1` becomes a fresh name (`x_2 = x_1 + 1`); each name has exactly one definition, so the compiler can trace any value back to where it was produced in one hop. That property is what makes constant folding, dead-code elimination, and “this [accumulator](#) stays in a [register](#) across the K-loop” cheap to prove. At branch joins, an artificial `φ` (phi) node picks which incoming SSA value to use. [Triton-IR](#) is SSA — that’s why `.asm['ttir']` reads `%5 = arith.addf %3, %4` instead of `x += y`.
- **Triton-IR (Part 2)** — hardware-agnostic [SSA](#) IR; the first lowering of your `@triton.jit` function. Inspect via `compiled.asm['ttir']`.
- **TritonGPU-IR (Part 2)** — GPU-targeted IR. Adds layout decisions: which values live in [registers](#) vs [shared memory](#), which [warp](#) owns which lane. Inspect via `.asm['ttgir']`.
- **LLVM IR (Part 2)** — the standard low-level IR; [TritonGPU-IR](#) lowers into it before becoming [PTX](#).
- **PTX (Parallel Thread Execution) (§6)** — NVIDIA’s virtual assembly. Stable across GPU generations; the [driver JITs](#) it to [SASS](#) at runtime. Inspect via `.asm['ptx']`.
- **SASS (§6, §9)** — the real machine code for a specific GPU (e.g., `sm_75` on [T4](#), `sm_90` on [H100](#)). What [cubin](#) contains. Not portable across architectures.
- **cubin (Part 2)** — the [SASS](#) binary the [driver](#) produces from [PTX](#). Cached, so the second launch is a dict lookup.
- **compute capability (`sm_XX`) (Parts 1, 4)** — the GPU’s architecture version. `sm_75` = [Turing](#) ([T4](#), RTX 20-series), `sm_80` = [Ampere](#) ([A100](#)), `sm_86` = Ampere consumer (RTX 30-series), `sm_90` = [Hopper](#) ([H100](#)). Higher = newer features ([TF32](#), FP8, transformer engine).

PERFORMANCE VOCABULARY

- **arithmetic intensity (AI) (Parts 3–4)** — FLOPs ÷ bytes loaded. Decides which side of the [roofline ridge](#) your [kernel](#) lives on. Vector add ≈ 0.08 FLOPs/byte; [FP32](#) matmul ≈ 8 ; [FP16](#) matmul on tensor cores $\approx 32+$.
- **roofline (Part 3)** — chart with [AI](#) on the x-axis, achieved [GFLOPS](#) on the y-axis. The bent ceiling is `min(peak FLOPS, bandwidth × AI)`. Left of the [ridge](#) = [memory-bound](#); right = [compute-bound](#).
- **ridge point (Part 3)** — the [AI](#) where the [roofline](#)’s bandwidth slope meets the compute ceiling. On [T4](#) [FP32](#), ~ 25 FLOPs/byte. Below the ridge, more compute can’t help; above it, more bandwidth can’t help.
- **memory-bound (Part 3)** — [kernel](#)’s ceiling is [HBM](#) bandwidth, not the math units. Vector add, [reductions](#), blur live here. Win by moving fewer bytes ([fusion](#), larger tiles, [coalescing](#)).
- **compute-bound (Parts 3–4)** — [kernel](#)’s ceiling is [FP32](#) or tensor-core throughput. Big matmuls live here. Win by feeding the tensor cores without starving them ([tiling](#), [autotune](#), [num_stages](#)).
- **memory wall (Part 3)** — the structural gap between what [HBM](#) can deliver and what the math units can consume. Has widened for 20+ years. [Fusion](#) exists to dodge it.
- **peak (all parts)** — the theoretical maximum a unit can do. Peak [FP32](#) FLOPS $\approx$ ([FMA](#) units) $\times$ clock $\times 2$ (since FMA is 2 FLOPs). Peak [HBM](#) $\approx$ (bus width) $\times$ (memory clock) $\times$ (channels). [T4](#): 8.1 TFLOPS [FP32](#), 320 GB/s. Real kernels at 60–90% are doing well.
- **GB/s (Part 3)** — gigabytes/second through [HBM](#). The metric [memory-bound](#) kernels are scored against.

- **GFLOPS / TFLOPS** (Parts 3–4) — billion / trillion floating-point ops/second. The metric [compute-bound](#) kernels are scored against.
- **FMA** (Parts 1–4) — *Fused Multiply-Add*: `a*b + c` with one rounding. Counts as 2 FLOPs. The [FP32](#) fallback when tensor cores aren't eligible (`M, N, K < 16` or non-[FP16/BF16](#) inputs).
- **accumulator** (Part 4) — [register](#) that holds the running sum across a [reduction](#) or matmul K-loop. The matmul trick is keeping it in registers across iterations, never round-tripping [HBM](#).
- **partial / partial sum** (Parts 2, 3) — the slice of a global result one program computes. Combining partials is either a second [kernel](#) pass or a [host-side](#) `.sum()`.
- **reduction** (Parts 2, 3) — operation that collapses many values to one (`sum`, `max`, `argmax`). Single-block reductions stay in [registers](#); cross-block reductions need atomics or two passes.
- **fusion** (Part 3) — collapsing two [kernel launches](#) into one so an intermediate stays in [registers](#) / [shared memory](#) instead of round-tripping [HBM](#). The standard cure for [memory-bound](#) chains.
- **tiling** (Parts 3–4) — splitting a problem into `BLOCK_M × BLOCK_N × BLOCK_K` chunks so each chunk fits in fast memory and is reused there. The reason matmul climbs the [roofline](#).
- **stencil** (Part 3) — an operation that computes each output element from a fixed local *neighbourhood* of its inputs: a 3×3 average blur, a convolution filter, a finite-difference step. The blur in notebook 04 is the workshop's stencil. Stencils are the reason [halos](#) exist — a [tiled](#) stencil can't fill its border without reading just past the tile edge.
- **halo** (Part 3) — the thin border of input a [tiled stencil](#) must read from its neighbours to compute the tile's own edge (a.k.a. *apron* or *ghost cells*). A 3×3 blur needs a 1-pixel halo on every side. Halo cells that fall off the image are masked to 0 — the 2D generalisation of the [tail mask](#), with two boundary checks (rows and columns) where vector-add had one. Get the halo mask wrong and the edges go grey or NaN.
- **warm-up** (Part 2) — running a [kernel](#) once before timing. Pays the [JIT](#)-compile + [cubin](#)-load + cache-populate cost off the clock. Skip warm-up and your “first run” is 100× too slow.
- **median timing** (Part 3) — reporting the median time across N runs instead of mean. Median ignores outliers from GC pauses, OS interrupts, and cold caches.
`notebooks/shared/benchmark_utils.median_seconds` is the helper.

PROFILING TOOLS

- **CUPTI** (Part 3) — *CUDA Profiling Tools Interface*. NVIDIA's low-level performance-counter API. Both Nsight and Triton's [Proton](#) sit on top of it.
- **Nsight Compute** (Part 3) — NVIDIA's deep-dive single-kernel profiler. Asks “*how well* did this [kernel](#) run.” Not used in the workshop; named as the canonical reference tool.
- **Nsight Systems** (Part 3) — NVIDIA's timeline / multi-kernel profiler. Asks “*when* did things run and *what overlapped*.” Sibling to [Nsight Compute](#).
- **Proton** (`triton.profiler`) (Part 3) — Triton 3's built-in profiler. Wraps [CUPTI](#); emits a `.hatchet` trace.
- `proton.scope(...)` (Part 3) — context manager that carves a named region into the trace so time / [GFLOPS](#) / [GB/s](#) attribute to the right block of code.
- `.hatchet` (Part 3) — file format [Proton](#) writes. A hierarchical call-and-kernel tree.

- **proton-viewer** (*Part 3*) — CLI that prints time, [GFLOPS](#), [GB/s](#) per scope from a [.hatchet](#) trace.

TRITON-SPECIFIC KNOBS

- **autotune** (*Parts 3–4*) — `@triton.autotune`. Compiles every `Config`, benchmarks on the first call per `key=` tuple, caches the winner. Re-runs the bake-off whenever any value in `key=` changes.
- **num_warps** (*Part 4*) — [warps](#) per program (1–16). More warps = more parallelism inside one program, more [register pressure](#), lower [occupancy](#). [T4](#) matmul sweet spot is often 4 or 8.
- **num_stages** (*Part 4*) — software-pipeline depth (2–4). Overlaps a tile's load with the previous tile's compute. Bump 2 → 4 if a profiler shows the [kernel](#) stalled on loads; higher costs [registers](#) (each in-flight tile needs its own).
- **GROUP_SIZE_M** (*Part 4*) — reorders matmul program IDs so neighboring programs share rows of A in [L2](#). ~1.3× on [T4](#) from improved L2 hit rate. The “swizzle” / “L2 reuse” trick.
- **tail mask** (*Part 2*) — the `offs < n` [predicate](#) that gates out-of-bounds lanes on the last program. Forgetting it is the #1 way to corrupt memory. The 2D version is the [halo](#) mask in notebook 04.
- **CompiledKernel** (*Part 3*) — what `kernel[grid](...)` returns. Inspect IR via `.asm['ttir' | 'ttgir' | 'llir' | 'ptx']`.
- **specialization key** (*Part 3*) — what Triton hashes to decide if a launch is a cache hit: dtypes + every `tl.constexpr` value + pointer alignment + integer args known to equal 1.
- **TRITON_CACHE_DIR** (*Part 3*) — env var that overrides `~/.triton/cache/`. Set it when sharing a cache between machines; unset + `rm -rf` to force a fresh compile.

NUMERICS

- **FP32** (*Parts 2–4*) — 32-bit IEEE float (1 sign + 8 exponent + 23 mantissa bits). Default [accumulator](#) dtype for `tl.dot`, even when inputs are [FP16/BF16](#).
- **FP16** (*Part 4*) — 16-bit float (1 + 5 + 10). Tensor-core input; ~8× the [FP32](#) throughput on [T4](#). Narrow dynamic range (max ≈ 65,504) — overflow risk on un-scaled sums.
- **BF16** (*Part 4*) — *bfloat16* (1 + 8 + 7). Same 8-bit exponent as [FP32](#) (so dynamic range matches); mantissa narrower. Tensor-core eligible from [Ampere+](#); safer than [FP16](#) for training.
- **TF32** (*Part 4*) — [Ampere+](#) only (not [T4](#)). 1 + 8 + 10 bits — [FP32](#)'s range, [FP16](#)'s mantissa. Tensor cores treat FP32 inputs as TF32 *by default* unless you opt out with `input_precision='ieee'`.
- **input_precision='ieee'** (*Part 4*) — `tl.dot` argument that forces strict [FP32](#) fidelity on [Ampere/Hopper](#) by disabling the [TF32](#) narrow path. No-op on [T4](#) (no TF32 hardware) but harmless.
- **softmax** (*Parts 3, 4*) — `softmax(x_i) = exp(x_i) / Σ exp(x_j)`. The workshop's running [memory-bound](#) example: naive softmax needs ~6N bytes of [HBM](#) traffic for ~3N FLOPs (load to max, load again to exp, load again to divide). [Fusion](#) compresses that to one pass.

GPU ARCHITECTURES AND PRODUCTS

- **T4** (*all parts*) — the Colab free-tier GPU. [Turing](#), `sm_75`, 40 [SMs](#), 320 GB/s [HBM](#), ~8.1 TFLOPS [FP32](#), ~65 TFLOPS [FP16](#) via tensor cores. Every “% of peak” number in the workshop is “% of T4.”

- **Turing** — NVIDIA's 2018 architecture. [T4](#), RTX 20-series. First generation with programmable tensor cores.
- **Ampere** — 2020. [A100](#), RTX 30-series. Added [TF32](#) (auto-narrow tensor-core path for [FP32](#) inputs) and [BF16](#).
- **Hopper** — 2022. [H100](#). Added FP8 (e4m3 and e5m2), the transformer engine, distributed [shared memory](#).
- **A100 / H100** — data-center GPUs ([Ampere](#) / [Hopper](#)). Multi-thousand-dollar; relevant because Triton kernels written for [T4](#) run on them unchanged — they just hit higher peaks.